

大規模言語モデル(LLM)・生成AI活用ガイド — 研究・実務:ループエンジニアリング含む

A Practitioner's Guide to Using Large Language Models and Generative AI in Economic History

Andreas Ferrara(ピッツバーグ大学) / NBER Working Paper No. 35374(2026年6月)

解説・演習構成:川口 有一郎

02

INTRODUCTION

背景 — なぜ経済史でLLMか

経済史はデータ集約的で、新聞・写真・地理記録・絵画など多様な資料を扱う。

従来は専門助手の雇用や機械学習・コンピュータビジョン・NLPの習得という高い参入障壁があった。

LLM/生成AIはこの障壁を大幅に下げ、専門技能のない研究者にも創造性の発揮を可能にする。

40%→<1%

解析エラーの削減例

630k

分類対象の絵画枚数

生成AI = 音声・動画・テキスト・画像・コードを生成するモデルの総称。

LLM = そのうちテキスト生成に特化した部分集合。

LLMの限界とリスク



ハルシネーション

参考文献の捏造、誤った歴史的事実・データの生成。自信を持って誤答する。



苦手なタスク

文献検索・歴史的事実の参照・空間推論/ジオコーディング・因果推論設計の生成。



ノイズの多い代理変数

生成された尺度はプロンプトやモデル次第で回帰係数の大きさ・符号すら変わる。



思考の外部委託

過度な依存は判断力を損ない、分野の思考の多様性を失わせる。執筆は研究者が担う。

反復的なLLM対話のワークフロー(表1)

0 調査

利用可能なLLM・費用を把握

1 課題提示

タスクとデータを例示し選択肢を求める

2 比較

各選択肢の長所・短所・適合性

3 理解

手法の詳細・前提・問題点

4 実装

小規模パイロットでコード生成

5 検証

正確性・効率性を批判的に確認

6 実行

出力サンプルをフィードバック

7-8 妥当性→拡張

検証データと照合し本格実装へ

LLMを使う4つの方法

チャットウィンドウ

ブラウザ上の対話。着想・手法説明・執筆に。プロジェクトフォルダとAGENTS.mdで文脈共有。制約はアップロード上限。

IDE統合型アシスタント

IDE=統合開発環境(Integrated Development Environment)。VS Code等でインライン補完・チャット。分析コード執筆に最適。

エージェントイック

Claude Code / Codex 等。自機のファイルを読み書き・実行し自律的に作業。研究助手への委任に最も近い。

APIモデル

API=アプリケーション・プログラミング・インターフェース。数千〜数百万文書へ同一処理を大規模実行。バッチで低コスト。

実例①:絵画の感情分類

設定: 1400年以降の絵画から8感情(満足・愉快・興奮・畏敬・恐怖・怒り・悲しみ・嫌悪)を抽出。CV知識なしの研究者を想定。

ゼロショットVLM(視覚言語モデル)を選択。ChatGPT-4o と OpenCLIP を実装。

約8万枚で訓練する二段階CNN(数週間～数ヶ月)に対し、LLMワークフローでは数分でコード生成。

結果

- ChatGPT-4o:8枚中7枚を正解
- CLIP:8枚中6枚を正解
- 630k枚の分類は約\$2,290、バッチで半減
- CNNは再現可能で大規模展開に優位

実践ガイド①:適否とデータ機微性

LLMは適切なツールか

- 文献検索は捏造多発 → Scholar/EconLit/JSTORへ
- 歴史的事実・データの参照は誤答リスク
- 空間推論は弱い → 確立ライブラリを呼ぶコードを書かせる
- 因果推論設計の生成は不可。既存設計への反論に使う

データの機微性

- データ利用契約が制約。API送信は第三者開示になりうる
- IPUMS氏名やProQuest等は送信不可
- 解決策①:ゼロデータ保持のエンタープライズ契約
- 解決策②:オープンウェイトモデルのセルフホスト

実践ガイド②:モデル選択・プロンプト・コスト

M

モデル選択

タスクに能力を合わせる。中位モデルで十分な場合も。推論モデルは多段タスクに。安価フィルター+高価分類器の連鎖。

F

ファインチューニング

OccCANINEは職業記述をHISCOへ96%精度。LoRA・量子化で消費者向けハードでも可能。

P

プロンプト設計

タスク定義・例示・ループリック(不確実オプション含む)・時代語彙・根拠。新RAに渡せるかがテスト。

\$

コスト管理

期間限定・事前選別・語周辺の切出し・ストップワード除去。キャッシングとバッチAPI(約50%減)。

実践ガイド③:検証と再現性

V

検証データ

手動コードで精度評価・プロンプト/モデル選択。選択用と最終精度用は分離。稀カテゴリは層化抽出+精度/再現率併記。

R

再現性とバージョンドリフト

温度0でも出力は揺れうる(乱数シードではない)。モデル識別子とアクセス日を固定し、生の出力をアーカイブ。

B

測定誤差のバイアス

生成尺度は非古典的誤差を含む代理変数。通常の標準誤差では見えない。プロンプト/モデル/尺度を変え頑健性を検証。

C

検定と補正

語除去テスト(GABRIEL)で内容読解を確認。検証サンプルでバイアスを推定・補正(ValidMLInference)。

応用領域と結論

構造化データ生成

スキャン表・手書き台帳・特許を構造化(誤り40%→<1%)

識別子なし結合

名前ブラインド国勢調査リンクで一致率43~62%

尺度の標準化

職業・技術・制度を時空間で調和。発明→普及の遅れ約10倍縮小

非標準データ

画像・音声・歌。FDR演説の感情分析、GPUで144分→8分

結論:守るべき4原則

タスクにツールを合わせる / 手動ベンチマークで検証する / 再生成できないものをアーカイブする

問い・判断・執筆は研究者の手元に残す。技術は流動的だがワークフローの原則は長く有効。

IDE統合型 — 「代替」でなく「組み込み」

答え: RStudioを「置き換える」のではなく、RStudioの中にLLMの機能を組み込んで一緒に使う。

2つの組み込まれ方

①

インライン補完

打鍵中にLLMが次行/ブロックを提案し、キー操作で採用・却下(初期例:GitHub Copilot)。

②

エディタ内チャット

開いているファイルについて質問でき、そのファイルが自動的に会話文脈に含まれる。

+

エージェンティックIDE

Cursor等は複数ファイルにまたがる編集もLLMに任せられる(次スライドへ連続)。

利点

作業中のファイルを常に把握。ローカル直接アクセスで、コピー&ペーストの手間とアップロード上限を解消。

位置づけ

チャット窓とは補完関係。着想はブラウザ、実装はIDE内アシスタント。RStudioは分析実行の場として残る。

エージェンティック — 「拡張」の本質は自律性

IDE統合型の延長だが、拡張の本質は編集範囲の拡大ではなく、LLMが自ら「行動」する自律性にある。

連続している面

- ローカルファイルへの直接アクセスは共通。
- CursorのようなエージェンティックIDEは両者の間で、複数ファイル編集が可能。
- その「複数ファイル編集」を推し進めた先がエージェンティック・コーディング・アシスタント。

質的に変わる面(本質)

実行

コードを実際に実行できる

判断

どの手順を踏むかをLLM自身が判断

委任

同じファイルシステム上で自律的に作業=作業を丸ごと委任

比喻:IDE統合型=「賢い会話相手」／エージェンティック=「能動的に働く研究助手」

ツール: Claude Code ・ Codex ・ Gemini CLI(デスクトップアプリ化) **注意:** 自律性↑=検証↑。DB消去事例あり。元データはフォルダ外に保管。

APIモデル — 「叩く」のは研究者側

答え: LLMが自動でAPIを叩くのではない。研究者のコードがLLMのAPIを叩き、LLMは呼ばれて応答を返す側。



研究者が指定するパラメータ

モデル版(例: gpt-4o-2024-05-13)／役割(system)／温度(創造性)。→ 一定の再現性を確保。

エージェンティックとの違い

エージェンティック=LLMが自ら手順判断・実行。API=LLMは受け身、制御は研究者のコード。

何をLLMに任せるか — アンケート例

誤解しやすい切り分け: LLMに任せるのは「数値集計」ではなく「意味の読み取り」を要する部分。

× LLMのAPIは不要

- 選択式設問の記述統計(平均・度数・クロス集計)
- pandas・dplyr等の通常ライブラリで完結
- むしろLLMは呼ぶべきでない(コスト増・数値は苦手で誤りうる)

○ APIモデルが活きる

- 自由記述回答など非構造データを各行ごとに判断
- 例: 回答を肯定/中立/否定に分類、トピック抽出
- 数式では書けない「意味の読み取り」がLLMの出番

正しいワークフローと注意点

アンケート自由記述を例にした、APIモデルの正しい流れ:

1

各行(回答)をループ

自由記述CSVをコーパスとして
ループ処理

2

LLMのAPIで分類

回答テキストを送り、感情/トピ
ックを分類

3

ラベルを書き出し

返ったラベルを解析し分類済み
CSVへ

4

通常コードで集計

ラベルの度数分布などを再びコ
ードで計算

5

研究者が確認

起動後は離席、完了後に出力を
確認

※ LLMに任せるのはステップ2のみ。集計(ステップ4)はLLMではなく通常のコードで行う。

検証を欠かさない

手作業でコード化した小さな検証サンプルで精度を確認してから全体に適用する。

コストを抑える

5,000件規模ならバッチAPIでコストを約半分に。1リクエスト=1行の.json/.jsonlで送信。

HANDS-ON EXERCISE

ループ・エンジニアリング演習

これまでの経済史の実例を出発点に、参加者それぞれが自分の業務や研究で回す「自分のループ」を設計します。対象は業務でAI活用を探る人全般です。

これまで(1~15枚)

経済史研究でのLLM活用の実例集



今日(この演習)

あなたの仕事に「自分のループ」を設計

当日の流れ

1 見つける

繰り返し直している/指示していることを1~2つ挙げる

2 載せる

その作業をLLMのループとして設計する

3 試す

小さく回し、検証しながら改良する

Lynch: Claude Codeを「チーフ・オブ・スタッフ」に

Jess Lynch(FoundersEdge 創業ゼネラルパートナー/プレシードVC) — 「10 Ways I Use Claude Code as My Chief of Staff」(川口先生の和訳付き)

何を作ったか

文脈切替(メール↔CRM↔資料)の消耗を、Claude Codeのマルチエージェント構成で解消。

1つの司令塔エージェントが、専門サブエージェント(各自の指示とツール権限)に振り分け。

接続先:Attio(CRM)・Gmail・Googleカレンダー/ドライブ・Airtable・Granola・ローカルファイル。

狙い:管理・運用の層を任せ、戦略・意思決定・人間関係に時間を回す。

代表的なワークフロー(全10種の一部)

- ・ メール→CRMの案件登録(重複確認・要約・資料整理まで)
- ・ 規模を保った個別フォローアップ(過去のやり取りを踏まえた下書き)
- ・ 関係インテリジェンス(横断検索で面談前ブリーフを生成)
- ・ 紹介の記録・会議準備・受信箱トリアージなどの小技

読みどころ:失敗させて指示を直す

手作業でやり直さず、エージェントの指示を修正して育てる(あるCRM連携は15の修正で精度が上がった)。うまくいった手順は記憶させる。

Andrew Ng:3つのループ(ループ・エンジニアリング)

Andrew Ng(Stanford / DeepLearning.AI) — The Batch。0→1のプロダクト開発を回す「3つのループ」を、時間スケールの違いで整理。

分単位

エージェント・コーディングループ

仕様(と任意でeval)を与え、AIがコード生成→テスト→反復。数分ごとにビルド&テスト、人の介入なしで長く進む。

時間単位

開発者フィードバックループ

開発者が成果物を見て方向づけ。自己テストが進み、QAより「どの機能を/UIをどう」といった高次の判断へ。

日～週単位

外部フィードバックループ

友人レビュー、αテスト、A/Bテストなど。ユーザーの反応が開発者のビジョン→仕様→エージェントへ還流。

読みどころ: 人間の役割は「文脈の優位(context advantage)」。AIが知らないユーザー・状況を知る限り、human-in-the-loopが要る。自分の仕事の「繰り返し直している/指示している」ことを1～2つ思い浮かべながら読む。

「自分のループ」の考え方

ループ=同じ指示を繰り返し与えている作業を、一度きちんと設計して自動的に回す仕組み。

1

入力

何を渡すか(メール・CSV・原稿など)

2

指示

毎回言っていること=プロンプト化

3

処理

LLMに任せる部分(意味の読み取り)

4

出力

欲しい形(分類済み表・下書き等)

5

検証

小さな正解サンプルで精度を確認

設計のコツ

- 小さく始める:1タスク1ループ。まず数件で試してから量を増やす。
- 任せるのは「意味の読み取り」。数値集計や単純作業は通常のツールで。
- AIスキルに応じて:チャット窓で試す→慣れたらエージェント/APIで自動化。

当日までに考えておくこと

次の2つの問いに、それぞれ1〜2つ具体例を用意してお越しください。そのまま演習の題材になります。

Q1

繰り返し「直している」ことは？

毎回同じ修正を加えている作業。例:定型メールの語調調整、表記ゆれの統一、フォーマット直し。

Q2

繰り返し「指示している」ことは？

人やAIに毎回同じ説明をしている作業。例:議事録の要約方針、レビュー観点、分類ルールの伝達。

持ち物: 上の具体例メモ + (あれば)その作業で使う実データやサンプル。当日は「見つける→載せる→試す」で自分のループを形にします。

LLM as system connector, not thinking brain

LLMは「考える脳」ではなく「システムを繋ぐ柔軟な接着剤」

1

いま起きている革命

実務(特にバックオフィス)で「API → LLM → ソフトウェアの民主化」が進行中。

2

Geminiの表現を借りれば

LLM=システムを繋ぐ柔軟な接着剤(例:SalesforceのAgentforce Operations)。

3

同時に生じる問い

LLMの仕組みから考えると、人間にとって代わるような業務は少ないのでは?(不動産業に限定せず議論)

以下は同じ問いをClaudeに投げた出力の整理。

判断基準は「検証コストの安さ」

活用の成否を決めるのは、モデルの賢さではなく、出力が正しいかを、どれだけ安く確かめられるか。

問う順序: 「LLMは何が得意か」ではなく「私の業務は検証が安い」を先に。

良い活用の四条件 — 四つ揃えばほぼ確実に成功、一つ欠けると怪しくなる

1

検証が安い

正誤が安く判定できる(コンパイルが通る/突合できる/人が5秒で分かる)

2

可逆である

間違いが取り消せる(下書きは捨てられる/送金は取り消せない)

3

入力がテキスト

入力がテキストとして存在する(なければ、まずそれを作る仕事)

4

正解が外部に

LLMを事実の源泉にしない。system of recordに委ねる

硬い骨格、柔らかい末端

設計原則(3つ)

1

制御フローは固める

経路が既知なら、コードで決定的に書く。LLMに選ばせない。

2

表現の可変性はLLMに開放

汚いPDF、バラバラの書式、口語の報告。RPAが死んだ場所=LLMの生きる場所。

3

事実は外部に置く

RAG・API・DB。生成させるのは形式であって内容ではない。

実務上の順序(5ステップ)

1

まず紙とプロセスをテキスト化する(地味な作業)

2

決定的に書ける経路を全部書き下す

3

残った「書き下せない箇所」だけをLLMに渡す

4

副作用の直前に人間を置く(承認ゲート。ログは事後で防止ではない)

5

エージェント(自律ループ)は③で経路が未知だった場合のみ検討

正しい期待値、そして一文で

個人の時間が浮くことと、コストが下がることは別

10人が1時間ずつ浮かせても、1人分の人件費は消えない。浮いた時間を回収する設計(別業務への再配置、頭数の削減、処理量の増加)がなければ、効率化は帳簿に現れない。

これは失敗ではないが、効率化でもない。CFOに説明するときは、この区別を守る。

一文にすると

LLMは「賢い意思決定者」としてではなく、**検証が安く、可逆で、事実を外部に持つ業務の、末端の形式変換器**として使うときに最も大きな価値を出す。その条件を満たす業務は、バックオフィスに大量に埋まっている。